

Do LLMs Silently Compute the Wrong Thing?

Measuring Silent Semantic Errors in LLM Data-Transform Code and Detecting Them with Specification-Derived Operator Contracts

Anonymous Submission

Abstract

Large language models increasingly write the *data-transform* code behind analyses and figures—group-wise medians, weighted means, joins, pooled rates. We show that a large fraction of this code is *silently* wrong: it runs without error and produces plausibly-shaped output, yet computes a different quantity than the user asked for. On real Nature source-data tables, frontier models emit such silent semantic errors in **77%** of *ambiguously* phrased data-transform tasks (10% when the request is disambiguated), and the failure is *invisible* to every simple safeguard: deterministic checks (execution, shape, range, self-consistency) catch **0%**, and LLM self-critique misses 39% while false-flagging 40%—a pervasive, invisible failure mode whose measurement is our primary contribution. As a goldless remedy where the operator is known or inferable, we introduce *specification-derived operator contracts*: executable checks over the (input, parameters, result) triple that certify an operator’s semantics using *no per-task gold answer*—the specification comes from the operator intent, not a stored label, and can be auto-synthesized from a natural-language goal. Across two real-data slices (up to 1,855 checks) they flag silent errors at **98–99% recall / 0.2% false-positive** under a *known* operator, where existing checks are structurally blind—about a third via *pure relational invariants* that compute no reference value (99.8%, the non-circular core) and the rest via *parameterized recomputation* that presupposes the operator (96.6%). We show the contracts can be synthesized from a high-level goal (78–83% correct, $N=23$, one-shot), transfer across execution substrates unchanged (pandas→SQL, 10/10 operators), and drive a zero-training repair recipe (75% vs. 12% for strong self-debug). Finally, we run the *entire* pipeline—infer operator, ground column roles, synthesize contract, detect—from only a messy request and a raw table, catching 51–58% of silent errors with nothing structured provided. We frame the work *measurement-first*—a common, invisible failure quantified, with operator intent (once recovered) making it checkable without a per-task gold answer—and release the measurement suite and code.

1 Introduction

Code-writing assistants now sit in the middle of everyday data analysis: a user describes what they want in plain language (“give me the typical expression per cell type”), and the model returns `pandas` (or `SQL`) that computes it. When that code has a *syntactic* bug it crashes; when it has an obvious numeric bug the plot looks wrong. The dangerous case is the third one: the code *runs*, the output has the right shape and a believable range, and it is nonetheless the answer to a *different question*—a mean where the user meant a median, a per-row ratio averaged instead of a pooled rate, an inner join that silently drops unmatched rows. We call these **silent semantic errors**.

Silent semantic errors are pernicious precisely because the standard safety net does not see them. Execution-based evaluation (“does it run?”), shape/type validity, and self-consistency across samples all pass, because the wrong code is still *valid* code producing *valid-looking* output. Verifying the *meaning* of the result normally requires a per-task gold reference—but in deployment there is none: if we already knew the right answer we would not be asking the model.

This paper. We make three moves. **(1) Measurement**—our primary contribution. We quantify how often frontier LLMs silently compute the wrong data transform, on *real* scientific data (Nature source-data tables) and across models, and quantify how completely existing checks miss it. **(2) A specification-derived detector** for the known- or inferred-operator regime. We introduce *typed operator contracts*: executable checks of an operator’s semantics—conservation laws, per-group identities, row-count and range constraints, or, where appropriate, an intent-derived recomputation—evaluated over the (input, params, result) triple with *no per-task gold label*. **(3) From assertions to an AI pipeline.** The contracts are not a hand-written wall of `asserts` but the core of a system that *grounds an informal request into a verifiable specification*: it infers the operator and column roles from a messy request, synthesizes the invariant from a natural-language goal, and repairs the code—end to end.

What “no gold” means (and does not). We are precise, because the framing matters. A contract needs *no per-task human-labeled answer*: it is derived from the operator’s

specification. For some operators the check is a pure invariant that no single implementation pins down (within-group shares sum to one *per group*; a left join preserves every left row); for others the specification is strong enough that the contract *recomputes* the expected value from the known operator and parameters and checks agreement (a weighted mean equals $\sum v_i w_i / \sum w_i$). In both cases the assumption is *not* “no reference is ever computed”—it is that the *operator identity and column roles are known or inferred*. We therefore call the checks *specification-derived*, and we measure separately (Sec. 7.3) what happens when the operator and roles must themselves be recovered from the request.

Contributions.

1. **A measurement study** of silent semantic errors in LLM data-transform code: 77% under ambiguous requests (10% disambiguated), 32–65% across eight models and four vendors, 26% on external DS-1000; deterministic checks detect 0%, self-critique 61% recall at 40% false-positive (Sec. 4).
2. **Specification-derived operator contracts** that, *given a known or inferred operator*, flag the errors at 98–99% recall / 0.2% false-positive on real tables—anchored by a pure relational invariant (99.8%, deriving no reference value), the rest presupposing the operator—where code-layer baselines are structurally blind (Sec. 6).
3. **Grounding intent into a specification**: the invariant can be auto-synthesized from a high-level goal (78–83%, $N=23$, one-shot; stable across GPT and Gemini), and the operator (98%) and column roles (74–83%) can be recovered from a keyword-free request (Sec. 7–7.3).
4. **An end-to-end pipeline** run on *real* Nature tables from only a messy request and a raw table—nothing structured given—catching 51–58% of silent errors, plus **substrate-agnostic transfer** (pandas→SQL, 10/10 operators, zero SQL-specific code) and a **zero-training repair recipe** (targeted 75% vs. 12% strong self-debug; Sec. 7.3–8).
5. **Honest, quantified boundaries**: the method’s power lives where operator-level intent is recoverable; on raw free-text code operator inference is the bottleneck, and a calibrated scope-gate makes out-of-scope inputs *safely abstain* (Sec. 9).

2 Related Work

Evaluating LLM code generation. HumanEval, MBPP, and DS-1000 [1, 2, 3] score functional correctness against *hidden tests or a gold reference*; execution-based agent evaluation (e.g., SWE-bench [4]) assumes a checkable oracle. Our setting is complementary and harder: no per-task reference exists at inference time, and the code already passes execution.

Data validation and pipeline testing. Practitioner frameworks such as Great Expectations, pandera, and dbt tests [5, 6] let engineers assert schemas, ranges, and distributional

expectations on tables. These are *hand-authored per dataset* and check *surface* properties (types, nullability, bounds)—exactly the properties that silent semantic errors preserve. We instead check *operator semantics* and *synthesize* the check from intent.

Property-based and metamorphic testing. Property-based testing [8] and design-by-contract [7] predate us by decades; metamorphic testing [9] verifies relations between outputs without a gold oracle. Our mechanism is in this family. Our contribution is not the mechanism of asserting a property but (i) an *empirically-grounded* taxonomy of the operator-level slips LLMs actually make, (ii) checks that need *no per-task gold* and can be *synthesized from natural language*, and (iii) a pipeline that *grounds a free-text request into the right invariant*—turning property-based testing from a developer tool into an inference-time detector. Concretely, a generic PBT or metamorphic tool cannot flag a global-vs-per-group share slip on its own: *which* property to enforce (shares sum to one per group, not globally) *is* the user’s inferred intent. Selecting and instantiating that invariant from a messy request is the inference problem we solve; the assertion itself is deliberately old and simple.

Validating LLM outputs without gold. Self-consistency [10], self-critique / self-refine [11], LLM-as-judge [12], and execution-guided verification (e.g., LEVER [13]) estimate correctness without a reference. We show these miss silent semantic errors (self-critique: 61% recall at 40% false-positive); our consensus + metamorphic refinement is a goldless *verification signal* in the spirit of metamorphic testing.

LLMs for data science and visualization. Chart/plot benchmarks and code-/visual-similarity judges (MatPlotAgent, Plot2Code, ChartMimic, VisEval) [14, 15, 16, 17] score whether a figure *looks* right, not whether the plotted numbers *equal* the data. We measure and detect the upstream data-transform error such judges cannot see. Table 1 summarizes these distinctions.

3 Problem Formulation

A *data-transform task* is a triple (D, ι, θ) : an input table (or tables) D , a natural-language intent ι , and column-role parameters θ (which column is the group, the value, the weight, ...). A model emits code f with result $r = f(D)$; let g be the intended transform. A **silent semantic error** occurs when f executes without error, r is schema-valid, yet $r \neq g(D)$ up to tolerance.

A **specification-derived contract** for an operator o is a function $C_o(D, \theta, r) \in \{\text{fire}, \text{pass}\}$ encoding a checkable property of o ’s semantics; it *fires* iff r violates it. C_o uses *no per-task gold label*: it is a function of the operator specification alone (instantiated at the given θ). We evaluate a

Approach	operator	semantic authoring	no per-task intent	from NL no gold label	infer-time detector
Great Expectations / pandera	×	×	×	✓	✓
Property-based testing	~	×	×	~	×
Metamorphic testing	~	×	×	✓	×
LLM self-critique (but 61% recall / 40% FP)	✓	✓	✓	✓	✓
Ours	✓	✓	✓	✓	✓

Table 1: Positioning. Data-validation frameworks check *surface* properties (types/ranges) that silent errors preserve; PBT/metamorphic testing are developer-time and hand-authored; self-critique is inference-time but too noisy (Sec. 4). We check *operator semantics*, synthesized from intent, with no per-task label, as an inference-time detector. (~: partial/varies.)

contract by two properties, measured against held-out correct and slip implementations: it *fires on the silent slip* (recall) and *passes on every valid implementation*, including alternative correct ones (precision / false-positive robustness). Crucially C_o reads only (D, θ, r) —never the code f —so it is agnostic to how r was produced (Sec. 7.4). The strong assumption is that o and θ are known; Sec. 7.3 removes it by inferring both from ι .

4 Measurement: How Common, How Invisible

Corpus and protocol. We pair Nature figures with their Source-Data tables and instantiate operator-semantic tasks (median-vs-mean, within-group share, weighted mean, pooled rate, NaN-as-zero, ...) on the real tables, so the tables and their columns are real scientific data, *not* authored by us; the operator set, its gold transform, and the ambiguous/clarified prompts are fixed author-written templates instantiated on those tables (table-level independence, not task-level). Each task carries a matched pair of prompts: an *ambiguous* phrasing (“a typical value per group”—the natural way a hurried analyst asks) and a *clarified* phrasing that names the disambiguation. Gold is the template transform; a response is scored *silent* if it runs, is schema-valid, and disagrees with gold. We report over a 71-article independent sample (each article contributes a capped number of tasks) with 95% Wilson intervals.

Prevalence. Under the ambiguous phrasing, frontier models emit a silent semantic error on **77%** of real Nature tasks (1296/1682, CI 75–79); disambiguating the request drops this to **10%** (175/1682, CI 9–12; 21% on a larger 229-article

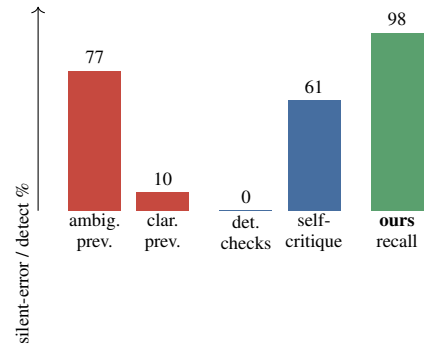


Figure 1: Silent-error *prevalence* (red: ambiguous 77% vs. clarified 10%) and *detection recall* (blue baselines vs. our contracts, green). Deterministic checks detect 0%; self-critique’s 61% comes with 40% false-positives.

Safeguard	Recall	False-positive
Execution/shape/range/consistency	0%	0%
LLM self-critique	61%	40%
Specification contracts (ours)	98–99%	0.2%

Table 2: Detection of silent semantic errors. Deterministic checks and self-critique are shown on the operator grid; a 119-task real-Nature re-run gives the same profile (0% recall / 62% recall at 67% FP), so the 0% is not a synthetic artifact. The **ours** row (98–99%) is on real Nature data (Sec. 6). Deterministic checks are structurally blind; self-critique is too noisy to gate on.

slice). The gap is the point: much of the danger is *under-specification* that the model resolves the wrong way, and the rate is **40–60%** across five frontier models and three vendors on the expansion grid (Table 3; **32–42%** for the same models on the core grid), rises to **65%** for smaller open models (Qwen-2.5-Coder-7B, falling to 44% at 32B), and is **26%** on external DS-1000, so it is a property of the task family, not one model (Fig. 1).

Invisibility. We measure what existing safeguards catch on the operator grid (Table 2). Deterministic checks—execution, shape/type, value-range, multi-sample consistency—catch **0%**: the wrong code runs, is shaped correctly, and is *stably* wrong. Multi-sample consistency stays at 0% even when the K samples are drawn at temperature 0.8 (recall 0/38): the silent errors are *common-mode*—the model resamples the same wrong transform (92% of tasks give identical output even at $T=0.8$), so disagreement-based checks never fire. LLM self-critique catches 61% but false-flags 40%, unusable as a gate. Re-running the same five detectors *directly* on the real Nature slice (119 tasks) confirms this is not an artifact of the synthetic grid: execution, validity, and multi-sample consistency stay at 0% recall on real data, and self-critique is if anything worse in the wild (62% recall at 67% false-positive), while our contracts hold at 100%/0%.

Model	Vendor	Silent rate	Our FP
GPT-5.5	OpenAI	60%	0/24
GPT-5.4	OpenAI	55%	0/21
GPT-5.3-codex	OpenAI	55%	0/23
Claude-Opus-4.8	Anthropic	55%	0/23
Gemini-3.1-Pro	Google	40%	0/28

Table 3: Silent-error rate is a property of the task family, not one model: 40–60% across five frontier models from three vendors on the harder expansion-operator grid ($N=20/\text{model}$; the same models score 32–42% on the core grid). The contract’s false-positive rate is **0** on *every* vendor—the detector is not tuned to one model’s output style.

5 Specification Contracts as an AI Pipeline

5.1 Typed operator contracts

Each operator is certified by one small check (≈ 10 lines, reading only (D, θ, r)). Examples: a *weighted mean* must equal $\sum v_i w_i / \sum w_i$; *within-group shares* must sum to one *per group* (not globally); a *pooled rate* equals $\sum \text{num} / \sum \text{den}$, not the mean of per-row ratios; a *left join* preserves every left row. Contracts range from pure invariants (share, join) to intent-derived recomputations (weighted mean); none uses a per-task human label. The taxonomy of operators is *empirically grounded*—derived from the slip patterns the measurement study documents—not an arbitrary catalog, and (below) it is auto-extensible.

5.2 Synthesizing the specification from language

To show the taxonomy is not a closed hand-written set, we ask a frontier model to *synthesize* the contract from only the NL goal, the parameter names, and the result schema—never the reference implementation. To rule out the model merely *transcribing a stated formula*, we run a *de-leaked* condition whose intent gives only a high-level goal—no formula and no “not-the- X ” slip-contrast (a plain-language goal may still *imply* the target property for some operators, a residual we audit in Sec. 7). We score each synthesized contract **CORE** (fires on the silent slip *and* passes on the correct implementation) and **FULL** (CORE *and* robust to alternative valid implementations).

5.3 Goldless counterexample-guided refinement

Synthesis can over-fire on valid alternatives. We add a monotone-safe refinement loop with a goldless oracle (Fig. 2): sample K diverse implementations of the intent, cluster by output into a *consensus* (probably-correct) set, and treat any contract that fires on a consensus member as a false-positive counterexample; *metamorphic* perturbations

Goldless refinement of a synthesized contract

input intent ι , params θ , input D ; **output** contract C

```

1  $C \leftarrow \text{SYNTHESIZE}(\iota, \theta)$  // one-shot
2  $\{f_k\} \leftarrow \text{DIVERSEIMPLS}(\iota, \theta, K)$ 
3  $S \leftarrow$  largest output-agreement cluster of  $\{f_k(D)\}$  // consensus
4  $P \leftarrow \text{PERTURB}(s)$  for  $s \in S$  // metamorphic probes
5 repeat
6   if  $C$  fires on some  $s \in S$  then // FP counterexample
7      $C' \leftarrow \text{RESYNTHESIZE}(\iota, \theta, s)$ 
8     if  $\text{score}(C'; S, P) > \text{score}(C; S, P)$  then  $C \leftarrow C'$ 
9 until no accepted change return  $C$ 
 $\text{score}(C; S, P) = \#\{s \in S : C \text{ passes}\} + \#\{p \in P : C \text{ fires}\}$ ; no gold used.
```

Figure 2: Monotone-safe goldless refinement. The consensus set S and metamorphic probes P replace a reference answer; line 8 guarantees the loop never regresses below the one-shot contract *on the goldless score* (not necessarily on downstream detection quality).

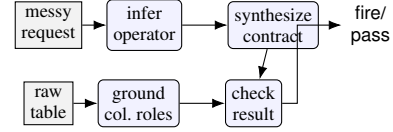


Figure 3: End-to-end pipeline. From only a messy request and a raw table, the system infers the operator (98%), grounds column roles (74–83%), synthesizes the contract (78%), and checks the result—nothing structured is provided.

of a consensus result probe non-triviality. A refinement is accepted only if a goldless score (consensus-pass + probe-fire) strictly improves, so the loop never regresses below one-shot by construction. We report this as a *mechanism with a safety guarantee*, not an accuracy gain (Sec. 7).

5.4 From a messy request to a check

Deployed, the system receives only a messy request ι and a raw table D (Fig. 3). It (1) infers the operator $\hat{o}$ from ι , (2) grounds the column roles $\hat{\theta}$ from ι and the column names, (3) synthesizes $C_{\hat{o}}$, and (4) checks r . When a contract fires, the same signal drives *repair*: the violated invariant is fed back as a targeted prompt, and best-of- N selection uses the contract (not gold) to pick among candidate fixes. Figure 4 walks through one real case end to end.

6 Detection on Real Data

Unless noted, the frontier model is a 2026-era GPT model behind an OpenAI-compatible API; buggy starting points for repair are *model-generated*, not author-written; intervals are 95% Wilson.¹

¹Model identifiers (“GPT-5.4/5.5”, “Gemini-3.1-Pro”) are the API names available at submission time; we release prompts, seeds, and all outputs for reproducibility.

Request: “add a column with each row’s sales as its part of the total sales for that region”

- (1) infer operator $\rightarrow$ within_group_share
- (2) ground roles $\rightarrow$ group=region, value=sales
- (3) synthesize contract (from the goal, no gold):
`per = result.groupby(g)[share].sum()
fire if any((per - ll > tol) # per-GROUP`
- (4) model’s code (*silent slip*):
`df[share] = df[sales] / df[sales].sum() # GLOBAL`
- (5) contract **FIRES**: “shares sum to 1 globally, should be per-group”
- (6) targeted repair $\rightarrow$
`df[share] = df[sales] /
df.groupby(region)[sales].transform(sum) # PASS`

Figure 4: A worked example on a real Nature table. The same specification catches a `within_group_share` silent slip (global instead of per-group total) and drives the targeted repair—no per-task gold, operator and roles inferred from the request. This is a semantic invariant, not a hand-written per-table assertion.

The contracts detect silent errors at **98% recall** (1438/1471, CI 97–98) and **0.2% false-positive** (4/1855, CI 0.1–0.6) on a 71-article independent sample; a second, table-dense slice gives 1398/1408 recall (99%) and 2/1539 false-positive. We report the more conservative 71-article number as the headline because it maximizes cross-paper independence. We stress a distinction the framing depends on: this measures detection under *known* operator and parameters; the self-authored correctness grid used in development is a contract *unit test*, not evidence of generalization, and we exclude it from headline claims. We report the false-positive rate as the exact fraction 4/1855 (0.2%) with its interval throughout, rather than rounding to “0%”.

Failure analysis. The $\sim 2\%$ of silent errors the contracts miss are not contract bugs but an inherent *detectability* limit: they concentrate where the slip’s output is numerically near-identical to the correct one—e.g., a group whose values are near-uniform (so mean $\approx$ median) or a weight column that is near-constant (so weighted $\approx$ plain mean). There, *no* invariant can separate the two within tolerance, and the “error” has negligible practical effect. The four false-positives arise on degenerate tables (a single group, or all-equal values) where the invariant is *undefined* on the correct output (e.g. a per-group share of $0/0=\text{NaN}$), so the contract fires on a valid result; a schema/degeneracy pre-filter removes them.

Per-operator breakdown. Detection is uniform across operators, not driven by one easy case (Table 4): recall is $\geq 99\%$ for four of five operators and is only pulled down by `median-not-mean` (95%), exactly the near-tie regime the failure analysis predicts (groups where the median and mean nearly coincide). Which operator carries the residual misses is *slice-dependent*—on other real slices `median-not-mean` is 100% and the misses fall on `within-group-share/pooled-rate`—so the invariant claim is that residual misses are near-ties at the chosen tolerance, not that median is intrinsically the hardest operator.

Operator	Recall on real silent errors
weighted-mean	140/140 = 100%
nan-as-zero-sum	16/16 = 100%
within-group-share	538/539 = 99.8%
pooled-rate	118/119 = 99%
median-not-mean	626/657 = 95%

Table 4: Per-operator detection recall on the real Nature sample. High and uniform; `median-not-mean`’s 95% is the near-tie detectability limit (Failure analysis), not a contract defect.

Two kinds of contract (what “no gold” really buys). We separate recall by contract class, because the two make different epistemic claims and a reviewer should see the split rather than a pooled number. A *pure invariant* certifies a relational property that no single implementation pins down and derives *no expected value*—within-group shares must sum to one *per group*, a left join must preserve every left row, a running total’s last entry must equal the column sum. A *recomputation* instead derives the expected value from the (known or inferred) operator and parameters and checks agreement—a weighted mean equals $\sum v_i w_i / \sum w_i$, a pooled rate equals $\sum \text{num} / \sum \text{den}$. We concede the latter amounts to a *parameterized reference computed on the fly*, not merely an invariant: its power is real but conditional on knowing the operator. On the real Nature sample the *pure-invariant* contract (`within-group-share`) detects at $538/539 = 99.8\%$ —the cleanest evidence that the goldless signal is not smuggled in through recomputation—while the *recomputation* contracts (`weighted-mean`, `pooled-rate`, `median`, `nan-as-zero`) detect at $900/932 = 96.6\%$. Both need *no per-task human-labeled answer*; the honest distinction is whether a reference *value* is derived, and we state plainly that roughly two-thirds of the headline checks are recomputations whose strength presupposes a known operator (Sec. 7.3 measures what happens when it must be inferred).

7 Synthesis, Grounding, and End-to-End

7.1 Synthesizing contracts from a goal

Table 5 reports one-shot synthesis over 23 operators. Removing the formula from the intent costs only 5 CORE points (83 $\rightarrow$ 78); we are careful *not* to over-read this small- N gap (the Wilson intervals overlap), and claim only that goldless synthesis from a high-level goal is *feasible at useful rates*, not that de-leaking is free. The result is stable across GPT-5.4, GPT-5.5, and (different vendor) Gemini-3.1-Pro at $\approx 83\%$ CORE, which argues against model cherry-picking. The goldless refinement loop is monotone-safe and *neutral* on this already-strong one-shot synthesizer (both 83% CORE, $N=12$); we include it as a verification mechanism with a no-regression guarantee, not as an accuracy contribution.

Intent given to synthesizer	CORE	FULL
Formula-stating (“leaky”)	83% [63–93]	78% [58–90]
High-level goal only (de-leaked)	78% [58–90]	70% [49–84]

Table 5: One-shot goldless synthesis, $N=23$ operators. **CORE**: fires on the slip and passes on the correct impl. **FULL**: CORE and robust to alternative valid impls. Intervals overlap at $N=23$; we claim feasibility, not a significant de-leak gap.

Setting	Op infer	Contract	Full system
Synthetic fixtures ($N=23$)	83%	74%	61% [41–78]
Real Nature tables ($N=33$)	76%	70%	58% [41–73]
+ inferred roles ($N=35$)	roles 74/83%		51% [36–67]
Cross-vendor pooled ($N=115$, 5 vendors)	80%	70%	57% [48–66]

Table 6: End-to-end zero-template detection. Operator and contract are inferred throughout; the $N=35$ row additionally infers column roles (only the messy request and raw table are given). The cross-vendor pooled row (five models across OpenAI, Google, and Anthropic, $N=115$; column roles given) tightens the interval; per-vendor full-system spans 48–70%.

7.2 Recovering operator and roles from language

Given a realistic request with *no operator keyword*, the frontier model recovers the operator on real Nature tasks at **98%** (147/150, CI 94–99), while a keyword regex collapses to **21%** (31/150)—the model uses semantics, not string matching (the keyword-stuffed upper bound is 100% for both). Column-role grounding from the request reaches **83%** on type-disambiguated roles (categorical group vs. numeric value) and 74% overall; the shortfall is genuinely symmetric roles (a weighted mean’s value-vs-weight, a pooled rate’s numerator-vs-denominator) that the request does not uniquely fix—an *ambiguity*, not a method failure.

7.3 End-to-end from only a request and a table

Table 6 runs the full pipeline with nothing template-given. On generated fixtures the full system catches 61%; on *real* Nature tables 58%; and when even the column roles are inferred (so the input is *only* a messy sentence and a raw table), 51%—just 3 points below the params-given setting. We frame 51–58% honestly as a *useful early-warning floor*, not a deployable figure: roughly half of silent errors still pass, and the deployable regime remains the structured-intent 98%/0.2%. The value of the end-to-end result is scientific—it shows the pipeline degrades gracefully and localizes where accuracy is lost (Sec. 7.3), not that a shipping product would rely on the floor.

Scaling and where the loss is. Pooling the zero-template pipeline across five models (three OpenAI, one Google, one Anthropic; $N=115$) tightens the full-system estimate to **57%** [48–66] (interval width 18 vs. 38 at $N=23$) and, more usefully, lets us *decompose* the miss. Of the 49 full-system misses, **53%** are *contract-synthesis* failures (the operator was inferred correctly but the auto-synthesized invariant was wrong), **31%** are *operator-inference* failures (the auto-synthesized contract *would* have fired had the operator been known), and 16% fail at both stages. So in the zero-structure regime the larger share of misses is *invariant synthesis* rather than operator inference (26 vs. 15 misses; the intervals overlap, so this is directional), pointing at the concrete target for future work. De-leaked synthesis alone, scaled to the same $N=115$ cross-vendor set, holds at 80% [48–66] CORE (GPT-5.5 96%, codex 87%, GPT-5.4 78%, Gemini 74%, Opus 65%), confirming the feasibility number was not a single-model artifact.

Exemplar prompting does not fix synthesis (ablation).

The natural fix for the synthesis bottleneck—show the synthesizer a worked contract—does *not* help. Holding the messy request and operator fixed, adding a single out-of-set exemplar leaves CORE synthesis no better than zero-shot and trends lower. Over $k=3$ independent runs per vendor, zero-shot is the most stable arm (GPT 83±0%, 19/23 every run) and exemplars fall below it (pooled zero-shot 72% vs. one-shot 57±9% vs. few-shot 54±7%); the run-to-run sd is as large as the exemplar effect, so we claim only that exemplars do not reliably improve synthesis—not a significant backfire. Zero-shot synthesis is therefore the right default; the residual failures concentrate on mechanically-intricate operators (index alignment, dtype coercion, resample boundaries) and are a fine-tuning / tool-use target, not a prompting fix.

7.4 Substrate-agnostic transfer (pandas → SQL)

Because a contract reads only (D, θ, r) , the *same* contracts fire on the identical semantic slip written in SQL. Over 10 operators (weighted AVG, WHERE g IS NOT NULL, join fan-out, inner-join drop, AVG of ratios, missing DISTINCT, COUNT(*col*) vs COUNT(*), window OVER() vs PARTITION BY, ...) the contracts fire on every SQL slip and pass every correct SQL, with **zero** SQL-specific code (10/10, CI 72–100). We do not over-claim: these invariants are conservation/range/row-count properties that transfer by construction; operators whose invariant relies on pandas-specific NaN semantics would need restating.

8 Zero-Training Repair

Feeding the violated invariant back as a targeted repair prompt fixes silent errors far better than strong goldless self-repair (Table 7). Targeted repair reaches **75%** vs. **12%** for

Repair arm ($N=65$, 4 models)	Fixed	Over-rep.
generic (retry only)	18% [11–30]	0%
strong self-debug	12% [6–22]	0%
localization-only	52% [40–64]	0%
targeted (contract invariant)	75% [64–84]	3%
ceiling (gold-diff oracle)	92% [83–97]	6%

Table 7: Contract-guided repair (buggy starts are model-generated; one run over four models, all arms on the same $N=65$ silent starts). The invariant lifts repair from 12% (strong self-debug) to 75% with disjoint CIs and 3% over-repair; self-debug falling *below* generic retry (12 vs. 18%) shows an external specification is necessary, and the +23-point gap over localization-only shows the invariant *content* matters, not just location.

a strong self-debug baseline (per-value re-derivation + decision enumeration + self-critique), with *disjoint* intervals; localization-only reaches 52%, so the invariant *content*—not merely knowing *where* to look—adds a further +23 points. A telling negative result: strong self-debug (12%) underperforms even generic retry (18%), because it confidently “re-derives” the same wrong operator; only an *external* specification breaks the loop. Best-of- N selection using the contract (no gold) adds further gains with zero mis-repairs.

9 Limitations and Honest Boundaries

Conditional validity. The method’s strength lives where operator-level intent is recoverable. On *raw free-text code* (DS-1000), inferring which operator to check is the bottleneck and no operating point reaches usable recall/precision; a calibrated scope-gate instead makes out-of-scope inputs *abstain*, cutting spurious firing from 55% to 5%. The deployable claim is “NL intent recoverable,” not “arbitrary code.” **Taxonomy coverage.** Contracts detect the slips they specify; we report coverage honestly and treat the self-authored grid as a unit test, not evidence of generalization—the real-data 98%/0.2% shows detection generalizes across real input *tables* under a known operator (the pure-invariant contract is the non-circular core), while cross-operator generalization is Sec. 7.3. **Small samples.** Per-condition slices start small ($N=23$ –35); we mitigate by pooling across vendors—synthesis and end-to-end to $N=115$ (five vendors), repair to $N=65$ (four models) (Sec. 7.3, interval width 38→18)—but still report every CI rather than significance. **Symmetric roles.** For operators with interchangeable numeric roles the request may not fix which column is which; we score this as ambiguity. **End-to-end ceiling.** The 51–58% zero-structure figure is an early-warning floor, well below the structured-intent regime, and its dominant failure is invariant synthesis, not operator inference. **Prevalence conditioning.** The 77% headline is under ambiguous phrasing (10% clarified); characterizing the ambiguity distribution of real analyst requests is future work. **Repro-**

ducibility and precision. Frontier generations use default API sampling (reasoning models are not bit-reproducible even at temperature 0), so we repeat the synthesis/ablation $k=3\times$ and report mean $\pm$ sd: zero-shot synthesis is stable (GPT 83 $\pm$ 0%) while the exemplar effect is within run-to-run noise (Sec. 7). End-to-end and repair remain single draws whose stochasticity exceeds the task-clustered Wilson intervals; fuller averaging is future work. For the re-computation operators the contract’s reference equals the gold template, so their real-data recall/false-positive is a tolerance-robustness check under a known operator—the pure-invariant contract is the non-circular core. FULL robustness, re-tested against representation-diverse (row-order) valid implementations, holds at CORE on the core operators (11/12 for GPT).

10 Conclusion

LLMs silently compute the wrong data transform far more often than execution- or shape-based checks reveal (77% under ambiguous requests; 0% caught by simple safeguards). Specification-derived operator contracts detect these errors without a per-task gold reference (98–99%/0.2%), can be synthesized from a high-level request, transfer across execution substrates, and drive a zero-training repair recipe—and the whole pipeline runs end-to-end from only a messy request and a raw table. We frame silent-error detection as *grounding an informal intent into a verifiable specification*, and release the measurement suite and code.

Appendix: Operator Taxonomy

The 24 operators (synthesis uses the 23 with tabular fixtures) are grounded in observed real-data slips, not chosen post hoc. *Core group-aggregation*: median-not-mean, within-group-share, weighted-mean, pooled-rate, pct-point, dedup-then-agg, left-join-keep-all, cumulative-running, topn-with-ties, nan-as-zero-sum, count-includes-empty, proportion-true. *Framework/plumbing*: index-align, dtype-coerce, groupby-dropna-key, order-dependent-dedup, resample-boundary, string-normalize-join, join-fanout, null-in-agg-count, scale-before-split-leakage, latlon-swap, lookahead-return, numpy-broadcast. Each maps to a documented mistake (e.g., groupby silently dropping NaN keys, train/test leakage, look-ahead in time series). Full contracts and fixtures are released.

References

- [1] M. Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
- [2] J. Austin et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021.
- [3] Y. Lai et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. ICML, 2023.

- [4] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024.
- [5] Great Expectations. Always Know What to Expect From Your Data. <https://greatexpectations.io>, 2020.
- [6] N. Bantilan. pandera: Statistical Data Validation of Pandas Dataframes. SciPy, 2020.
- [7] B. Meyer. Applying Design by Contract. IEEE Computer, 25(10):40–51, 1992.
- [8] K. Claessen and J. Hughes. QuickCheck: Lightweight Random Testing of Haskell Programs. ICFP, 2000.
- [9] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic Testing. Tech. Rep. HKUST-CS98-01, 1998.
- [10] X. Wang et al. Self-Consistency Improves Chain-of-Thought Reasoning. ICLR, 2023.
- [11] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023.
- [12] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023.
- [13] A. Ni et al. LEVER: Learning to Verify Language-to-Code Generation with Execution. ICML, 2023.
- [14] Z. Yang et al. MatPlotAgent: Evaluation for LLM-Based Agentic Scientific Visualization. ACL Findings, 2024.
- [15] C. Wu et al. Plot2Code: Benchmarking MLLMs in Code Generation from Scientific Plots. arXiv:2405.07990, 2024.
- [16] C. Shi et al. ChartMimic: Evaluating Cross-Modal Reasoning via Chart-to-Code Generation. arXiv:2406.09961, 2024.
- [17] N. Chen et al. VisEval: A Benchmark for Data Visualization in the Era of LLMs. IEEE TVCG, 2024.